

DEEP LEARNING FUNDAMENTALS

1. Deep learning - A subset of machine learning that uses multi-layer neural networks to learn complex patterns from data.

Machine learning

1. Often needs manual feature engineering.
2. Works well with smaller or structured datasets.
3. Models can be simpler.
4. Usually less compute.

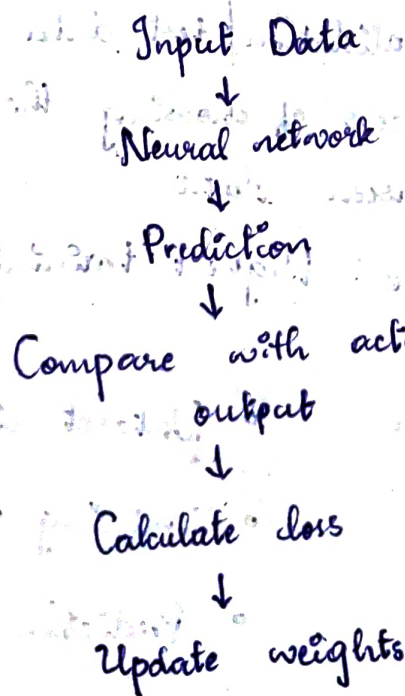
Deep learning

1. Learns many useful features automatically.
2. Especially strong for large and unstructured data.
3. Uses neural networks with multiple layers.
4. Usually needs more compute.

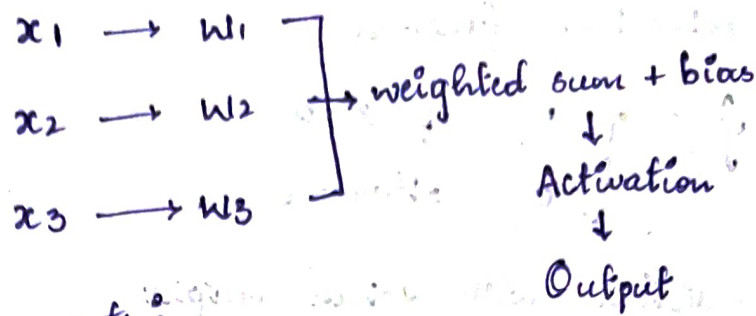
Applications :

- Image recognition
- Object detection
- Speech recognition
- NLP
- Recommendation Systems
- Generative AI

Basic Flow -



2. Artificial Neuron - The basic computational unit of a NN that combines inputs using weights and bias and then applies an activation function



Main Components:

- Input - Data entering the neuron
 - Weight - Importance given to an input
 - Bias - Extra adjustable value.
 - Activation Function - Decides the neuron's output
- are updated during training

Basic equation:

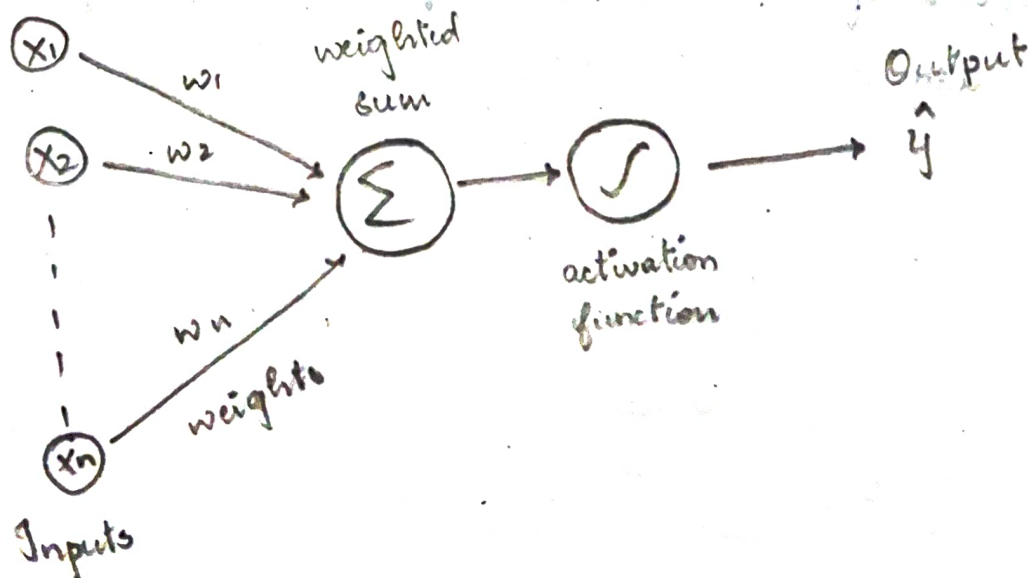
$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

then output:

$$\text{output} = f(z)$$

where $f \rightarrow$ activation function

3. Perceptron - A simple single-layer neural network used mainly for binary classification with linearly separable data.



Working -

1. Compute the weighted sum

$$z = \sum_{i=1}^n w_i x_i + b$$

2. Apply the activation function

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

3. Compare prediction with actual output

$$\text{error} = y - \hat{y}$$

4. Update the weights (learning rule)

$$w_i \leftarrow w_i + \eta (y - \hat{y}) x_i$$

5. Update the bias

$$b \leftarrow b + \eta (y - \hat{y})$$

6. Repeat for all samples across multiple epochs

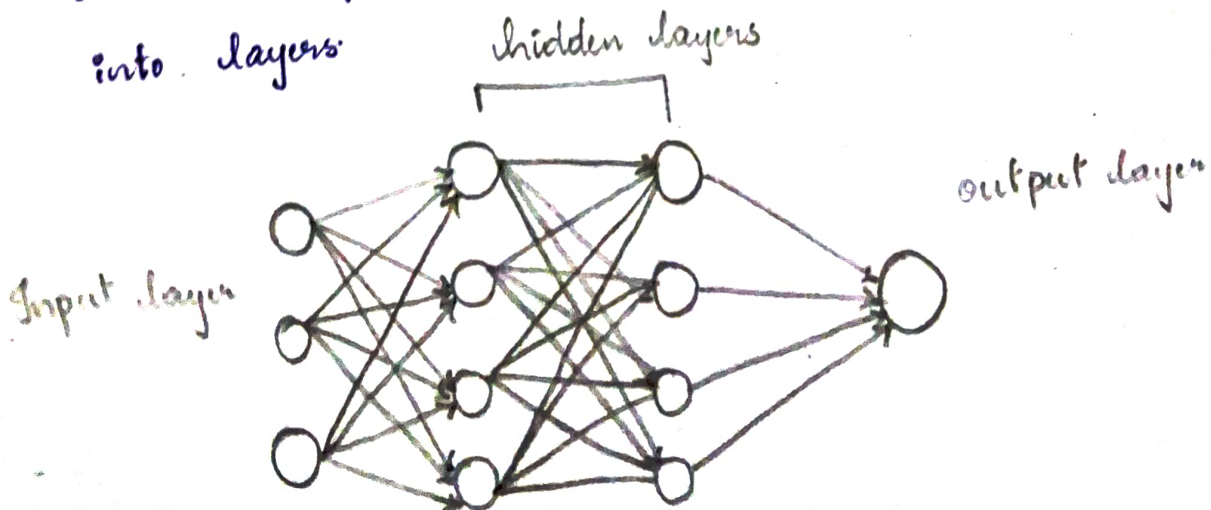
7. Final learned model

$$\hat{y} = \text{step}(w^T x + b) = f(w^T x + b)$$

Limitation - Can only learn linear decision boundaries.

4. Artificial Neural Network - ANN

A network of interconnected artificial neurons organized into layers.



Input layer - Receives input features

Hidden layer - Learns patterns and representations from the data.

Output layer - Produces the final prediction.

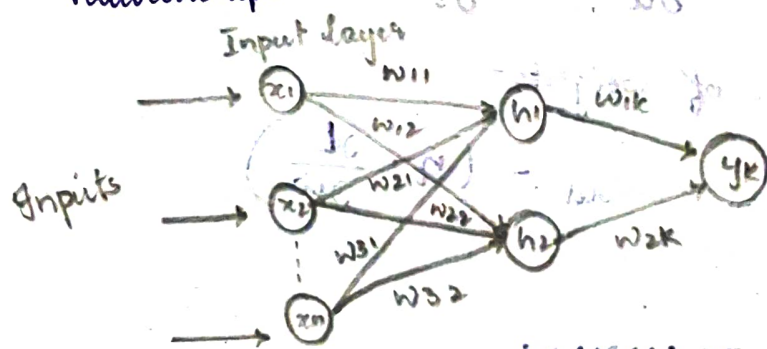
3. Multi-layer Perceptron — MLP -

A feed-forward neural network containing an input layer, one or more hidden layers, and an output layer.

* MLPs are fully connected.

Why multiple layers?

→ Allow the network to learn more complex non-linear relationships.



Working -

1. Forward Propagation

$$z = \sum w_i x_i + b$$

Activation function

• Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$

• ReLU $f(z) = \max(0, z)$

• Tanh $\tanh(z) = \frac{2}{1+e^{-2z}} - 1$

2. Loss function

$$L = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i) \right]$$

for regression : $MSE = \frac{1}{N} \sum (y_i - \hat{y}_i)^2$

3. Back Propagation

minimize the loss function

i. Gradient calculation - gradients of loss function with respect to each weight and bias are calculated.

forward pass $\Rightarrow z^{(l)} = w^{(l)} a^{(l-1)} + b^{(l)}$

$$a^{(l)} = \sigma(z^{(l)})$$

ii. Error propagation

iii. Gradient Descent -

backward pass -

$$\frac{\partial L}{\partial w^{(l)}} = \frac{\partial L}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial w^{(l)}}$$

optimization of weights -

$$w_{new} = w_{old} - \left(\eta \cdot \frac{\partial L}{\partial w} \right)$$

L = loss function

NEURAL NETWORK TRAINING

6. Forward Propagation - The process of passing input data through the nn to generate a prediction.

ACTIVATION FUNCTIONS

7. Activation Function - A function applied to a neuron's output that allows neural networks to learn nonlinear patterns.

Without activation functions $\rightarrow$ multi- nn layers would behave
 $\downarrow$
one linear transformation.

- **ReLU (Rectified Linear Unit)**: An activation function that outputs zero for negative values and keeps positive values unchanged.

$$\text{ReLU}(x) = \max(0, x)$$

$$x < 0 \rightarrow 0, \quad x > 0 \rightarrow x$$

Commonly used in: Hidden layers.

Limitation: Some neurons can become inactive if they continuously receive negative values.

- **Sigmoid**: An activation function that converts a value into a number between 0 and 1.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

O/p - 0 to 1

Commonly used: Binary classification output layer

0.91 $\rightarrow$ $\uparrow$ probability of class 1

0.12 $\rightarrow$ $\downarrow$

Limitation: can suffer from vanishing gradients in deep hidden layers.

- **Tanh**: An activation function that maps values btw $(-1, 1)$

$$\tanh(x)$$

O/p range: -1 to 1

Historically common in neural network and recurrent n.

- **Softmax**: An activation function that converts output scores into probabilities across multiple classes.

eg: Cat $\rightarrow$ 0.70

Dog $\rightarrow$ 0.20

Bird $\rightarrow$ 0.10

Probabilities sum to 1

Commonly used : Multiclass classification %p layer

Memory -

sigmoid $\rightarrow$ 0 to 1

tanh $\rightarrow$ -1 to 1

sigmoid $\rightarrow$ binary class

softmax $\rightarrow$ multiclass "

LOSS FUNCTIONS

8. Loss Function - Measures how far the model's prediction is from the actual target.

Training tries to minimize loss:

• MSE = $\frac{1}{n} \sum (y - \hat{y})^2 \Rightarrow$ Regression use

• Binary Cross-Entropy - A loss function used for binary classification

Used when %p is 1

0/1, Yes/No, Spam/Not Spam

• Categorical Cross-Entropy - A loss function used for when predicting one class from multiple categories

example : Cat, Dog, Bird

Problem	Output Activation	Common Loss
Regression	Linear	MSE
Binary class	Sigmoid	BCE
Multiclass class	Softmax	Categorical CE

BACK PROPAGATION

9. Backpropagation - The process of sending the prediction error backward through the network to calculate how each weight contributed to that error.

Forward :

Input $\rightarrow$ prediction

Backward :

Error $\rightarrow$ Weight updates

$\rightarrow$ Gradient - A value that tells us how much and in which direction a parameter should change to reduce the loss.

$\rightarrow$ Chain Rule - Backpropagation uses the chain rule from calculus to calculate gradients across multiple layers.

GRADIENT DESCENT

10. Gradient Descent - An optimization algorithm that updates model parameters in the direction that reduces the loss.

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

w = weight

L = loss

η = learning rate

$\rightarrow$ learning rate : A hyperparameter controlling how large each weight update is during training.

1. Batch Gradient Descent : Calculates the gradient using the complete training dataset before making 1 update.

$$\theta^{(k+1)} = \theta^{(k)} + \alpha \nabla J(\theta^{(k)})$$

θ = vector of (weights & bias) to optimize

k = current iteration index

α = learning rate

$\nabla J(\theta)$ = something like compass that points uphill toward the error.

Advantage: Stable updates

Limitations: Slow for very large datasets

2. Stochastic Gradient Descent - SGD: Updates model parameters using one training example at a time.

$$\theta := \theta - \eta \cdot \nabla \ell(x_i, y_i; \theta)$$

$\nabla \ell$ = gradient operator

θ = parameter vector

$\ell(x_i, y_i; \theta)$ = loss function for only single randomly chosen instance.

Advantages: fast updates, can work with large data

Limitations: Updates can be noisy.

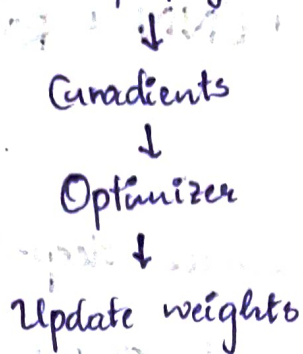
3. Mini-Batch Gradient Descent - Updates weights using small groups of training examples.

$$\theta = \theta - \frac{\alpha}{m} \sum_{i=1}^m \nabla \ell(\theta, x^{(i)}, y^{(i)})$$

OPTIMIZERS

10. Optimizer - An algorithm that uses gradients to decide how nn weights should be updated to minimize loss.

Backpropagation



SGD Optimizer	Momentum Opt	RMSProp Opt	Adam Optimizer
1. Updates params in direction opposite to gradient ⇒ gradient ↓ move wt toward lower loss 2. Advantages: - Simple - Memory efficient 3. Limitation: - converge slowly - can oscillate	1. Improves SGD by using info from previous updates to build movement in useful directions. 2. Advantages: - Faster convergence - reduces oscillation ⇒ Momentum ↓ SGD + previous direction	1. An optimizer that adjusts the α separately for diff parameters based on recent gradients. ⇒ RNN training → large gradients ↓ smaller effective updates * likewise - small	1. An optimizer that combines ideas from Momentum and RMSProp to provide adaptive and efficient wt updates Adam ↓ Adaptive Moment Estimation 2. Advantages: - Fast convergence - Adaptive α - default choice.

11: Epoch, Batch Size and Iteration

- Epoch: One complete pass through entire training set.
- Batch size: No. of training samples processed before one parameter update.
- Iteration: One parameter update using one batch.

Example -

Dataset = 1000 samples
Batch Size = 100
Iteration per Epoch = 10

So:

1 Epoch = entire set seen once

11/09/2026

TENSORFLOW + KERAS + CNN + RNN + LSTM + GRU + GAN + AUTOENCODERS

1. Tensorflow - An open-source deep-learning framework used to build, train and deploy neural-network models.

Main uses:

- Neural networks
- Time-series models
- Image classification
- Model deployment
- NLP

→ Tensor - A multi-dimensional array used by Tensorflow to store and process data.

SCALAR → 0D

VECTOR → 1D

MATRIX → 2D

TENSOR → 3D+

example:

import tensorflow as tf

x = tf.constant([[1, 2], [3, 4]])

2. Keras - A high-level API inside Tensorflow that makes building neural networks easier.

Model flow:

Define model

↓
Compile

↓
Fit

↓
Evaluate

↓
Predict

→ Sequential Model - Used when layers are arranged one after another.

```
→ model = tf.keras.Sequential([  
    tf.keras.layers.Dense(  
        64, activation = "relu",  
    ),  
    tf.keras.layers.Dense(  
        1, activation = "sigmoid"  
    )  
])
```

° Dense layer - A fully connected neural-network layer where every neuron connects to every neuron in the next layers.

* Important Keras Function

1. compile() - configures the model

```
→ model.compile(  
    optimizer = "adam",  
    loss = "binary_crossentropy",  
    metrics = ["accuracy"]  
)
```

2. fit() - trains the model

```
→ model.fit(X_train, y_train, epochs=10)
```

3. evaluate() - measures model performance

```
→ model.evaluate(X_test, y_test)
```

4. predict() - generates predictions.

```
→ model.predict(X_test)
```

COMPUTER VISION + CNN

3. Computer Vision - A field of AI that enables machines to understand and process images and videos.

Common tasks:

- Image classification
- Object Detection
- Image Segmentation
- Face Recognition
- Pose detection

4. Image Representation -

An image is represented as numbers.

• Grayscale Image - Height $\times$ Width

• RGB Image - height $\times$ width $\times$ 3

3 channels are: Red, Green, Blue

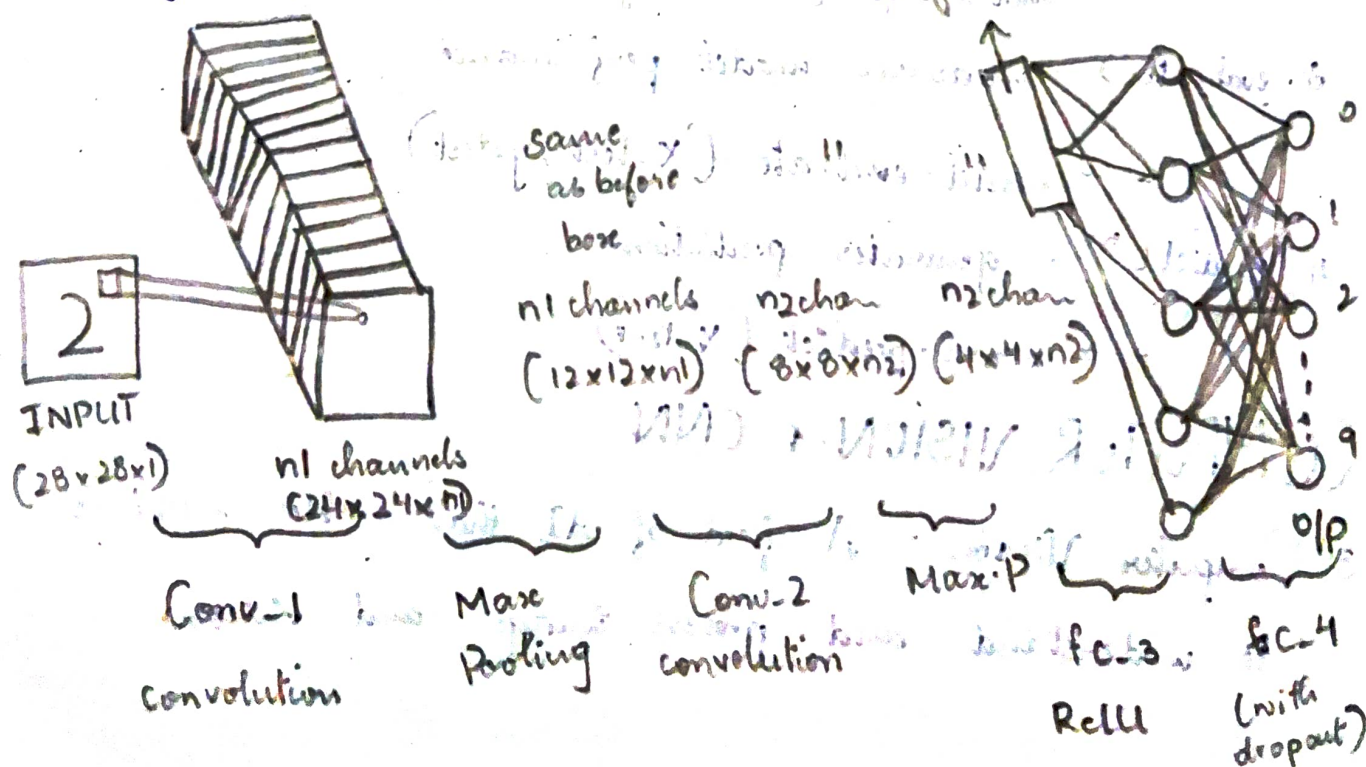
examples: $224 \times 224 \times 3$

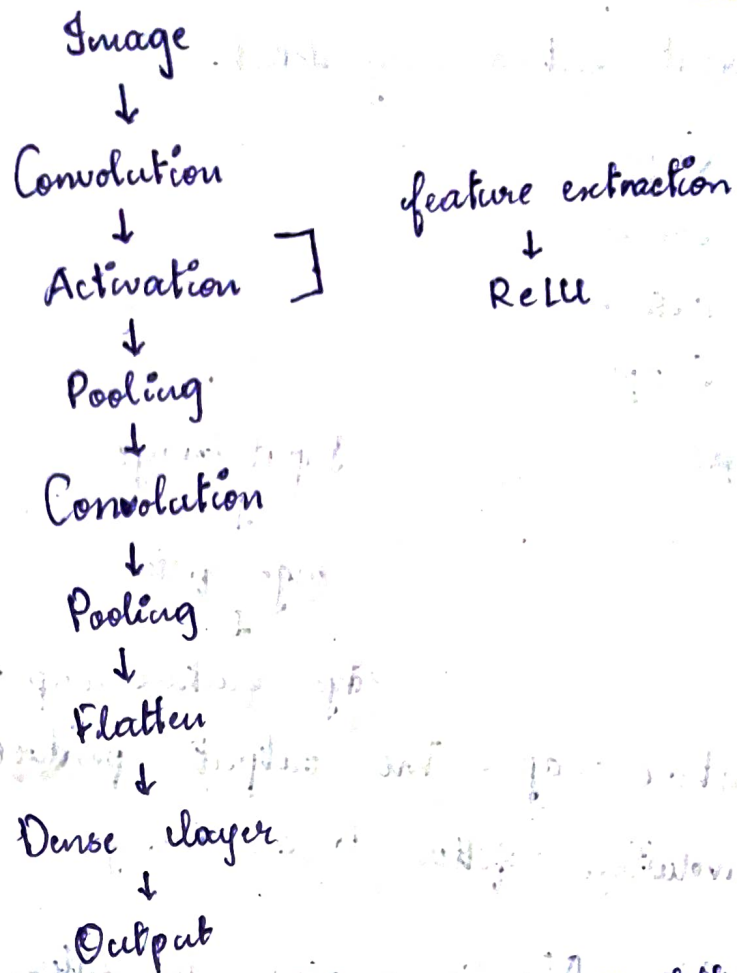
pixel values are usually: 0 to 255

often normalized to: 0 to 1

5. CNN - Convolutional Neural Network -

A neural network designed by mainly for image data that automatically learns spatial features such as edges, textures, shapes and objects.





→ **Convolution** - The process of sliding a small filter over an image to extract features.

example image :

1	2	3
4	5	6
7	8	9

filter :

1	0
0	1

The filter slides over the image and calculates weighted sums.

General convolution equation:

$$Y(i, j) = \sum_m \sum_n X(i+m, j+n) K(m, n)$$

Labels for the equation:
 $Y(i, j)$ is labeled 'output'
 $X(i+m, j+n)$ is labeled 'input image'
 $K(m, n)$ is labeled 'kernel / filter' (with an arrow pointing to the equation)

→ **Kernel / filter** - A small matrix used to detect specific image patterns.

Different filters may detect:

Edges

Corners

Textures

Shapes

example:

Input image



edge filter



edge feature map

↳ Feature map - The output produced after applying a convolution filter to an image.

↳ Stride - Number of pixels the filter moves at each step.

Stride = 1 → move one pixel

Stride = 2 → move two pixels

Larger stride:

smaller output

less computation

↳ Padding - Adding pixels around the border of an image.

2 common types:

◦ Valid padding - No extra padding, OP becomes smaller

◦ Same padding - adds padding so output dimensions are approximately preserved

Output-size formula:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

where, N = input size

F = filter size

P = padding

S = stride

↳ Pooling - Reduces the spatial dimensions of feature maps.

Main types: Average P, Max P

- Max pooling - chooses the maximum value.

example:

1	3
2	4
→ 4	

Benefits:

- reduces computation
- helps control overfitting
- keeps important features

↳ Flattening - Converts multi-dimensional feature maps into a one-dimensional vector.

CNN -

Early CNN layers learn - Edges

Middle layers learn - Textures / Patterns

Deep layers learn - Complex objects

Advantages:

- Automatically learns image features
- Fewer parameters than fully connected networks for images
- Very effective for visual tasks.

Limitations:

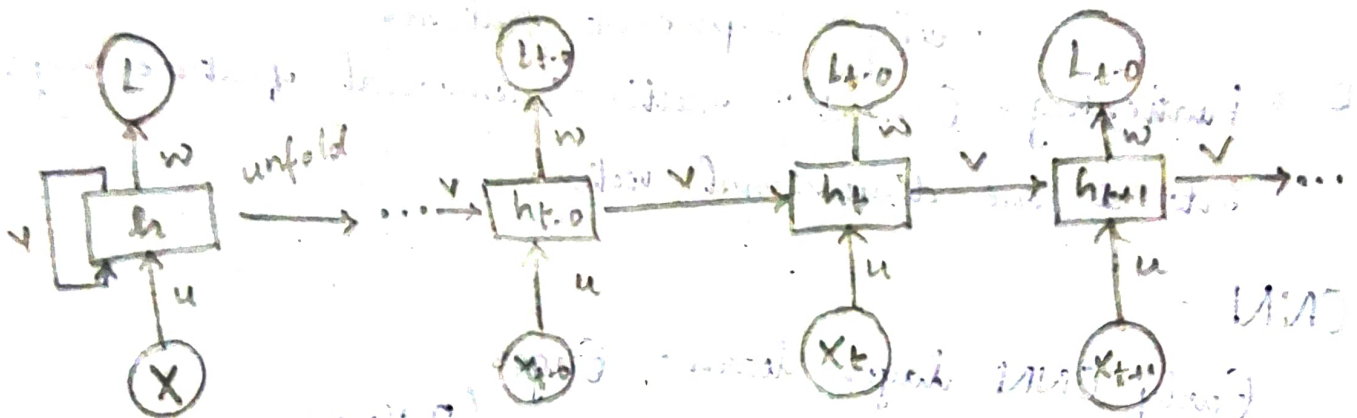
- Requires more computation
- Usually needs plenty of training data
- Harder to interpret

RECURRENT NETWORKS

6. RNN - A neural network designed for sequential data that uses information from previous time steps while processing the current input.

Used for :

- Text
- Time series
- Speech
- Sequences



Another view

Previous Hidden State



Current input $\rightarrow$ RNN $\rightarrow$ Current hidden state

$\rightarrow$ Hidden State - The memory of an RNN that carries information from previous time steps.

RNN equation -

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

output -

$$y_t = W_{hy} h_t + b_y$$

where x_t = current input

h_{t-1} = previous hidden state

h_t = current hidden state

RNN Working -

1. State update :

$$h_t = f(h_{t-1}, x_t)$$

h_t = current state

h_{t-1} = previous "

x_t = input at current time step

2. Activation function :

$$h_t = \tanh \left(\underbrace{W_{hh}}_{\substack{\downarrow \\ \text{wt matrix} \\ \text{for recurrent} \\ \text{neuron}}} \cdot h_{t-1} + \underbrace{W_{hx}}_{\substack{\downarrow \\ \text{wt matrix} \\ \text{for input neuron}}} \cdot x_t \right)$$

3. Output :

$$y_t = W_{hy} \cdot h_t$$

+
weight of output layer.

Types of RNN -

1. One-to-One RNNs

2. One-to-Many RNNs

3. Many-to-One RNN

4. Many-to-Many RNN

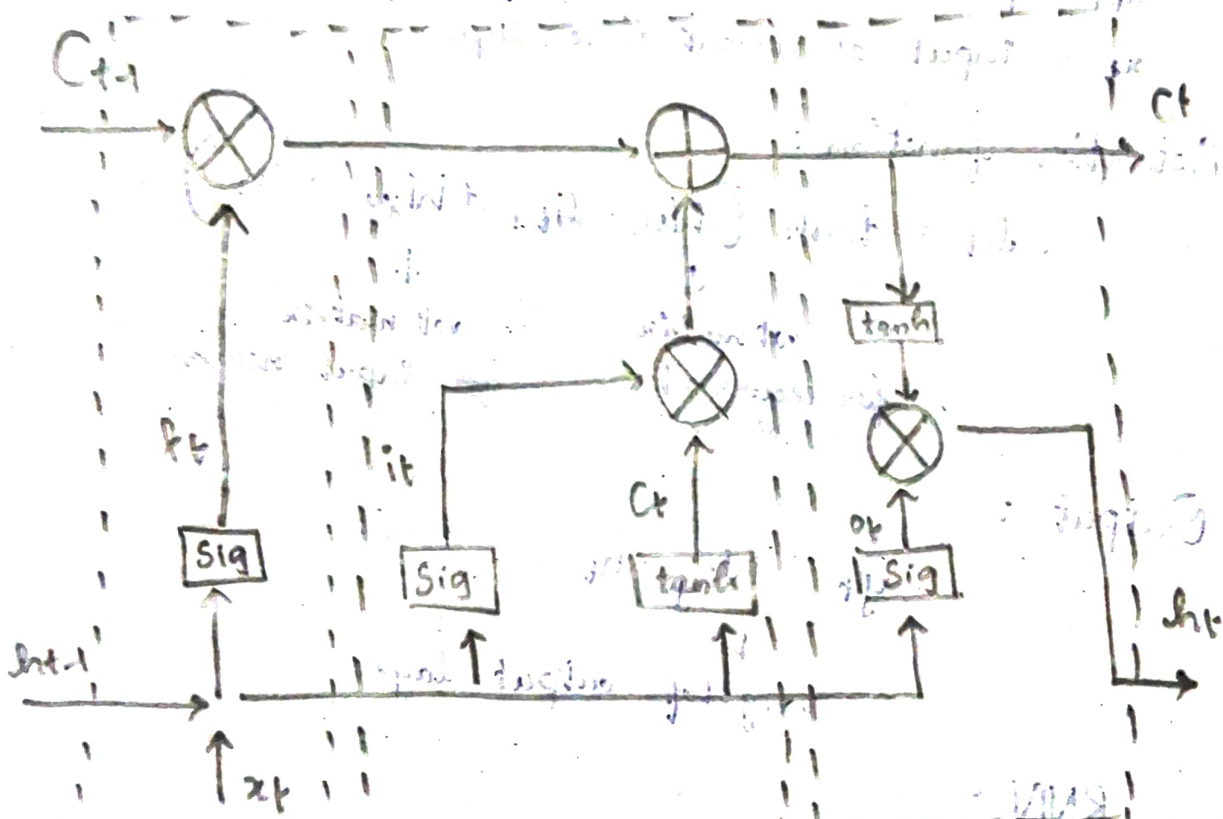
RNN Problem : ~~Vanishing~~ Gradient - During training, gradients may become extremely small when propagated across many time steps.

→ RNN forgets old information

7. LSTM - Long Short-term Memory - A special type of RNN designed to remember long-term dependencies using a memory cell and gates.

Components:

- Forget gate
- Input gate
- Cell State
- Output gate



• Forget gate & Decides what old info should be kept or removed

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

O/p range:

0 $\rightarrow$ forget

1 $\rightarrow$ keep

- Input gate: Decides what new information should enter memory.

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

$$\bar{C}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$$

- Cell State update: The memory cell is updated using:

$$C_t = f_t C_{t-1} + i_t \bar{C}_t$$

means: old useful memory + new useful information

↓
updated memory

- Output gate: Decides what part of memory becomes the current hidden output.

$$O_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = O_t \tanh(C_t)$$

Advantages:

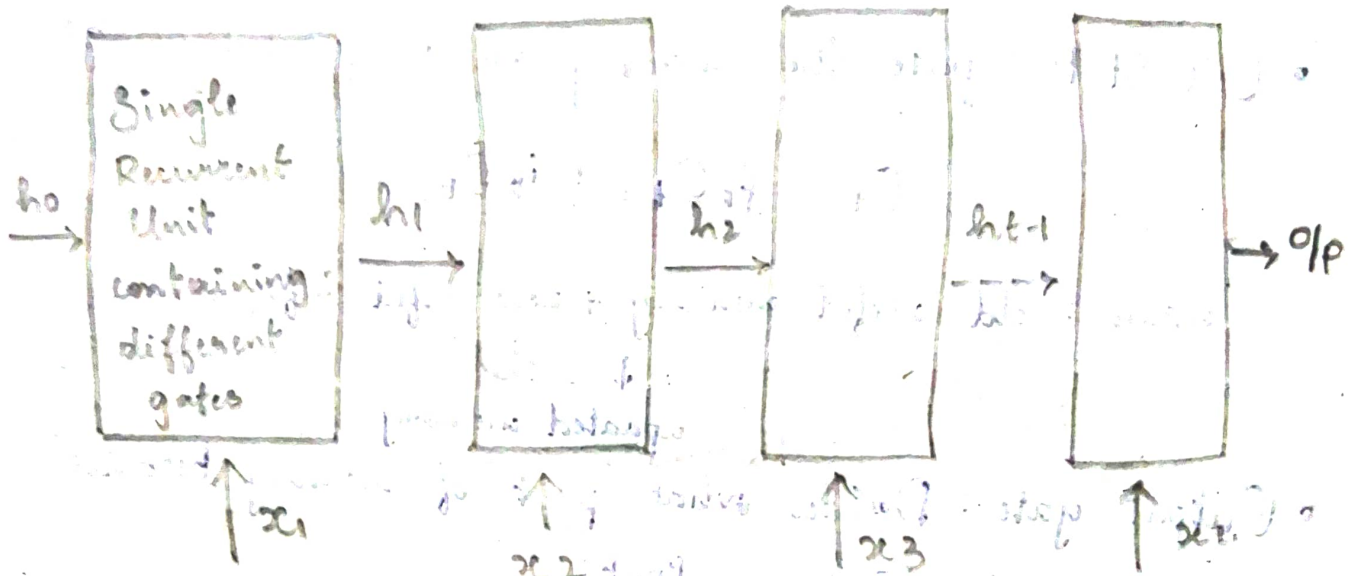
- handles long-term dependencies
- better memory than basic RNN
- useful for sequential data

Limitations:

- more parameters
- slower than simpler RNNs
- more complex architecture

8. GRU - Gated Recurrent Unit - A simplified recurrent network similar to LSTM that uses fewer gates and parameters.

→ Reset Gate → Update Gate



- Update gate: decides how much previous information should be retained.

$$z_t = \sigma(W_z [h_{t-1}, x_t] + b_z)$$

- Reset gate: Decides how much previous info should be ignored.

$$r_t = \sigma(W_r [h_{t-1}, x_t] + b_r)$$

Candidate hidden state:

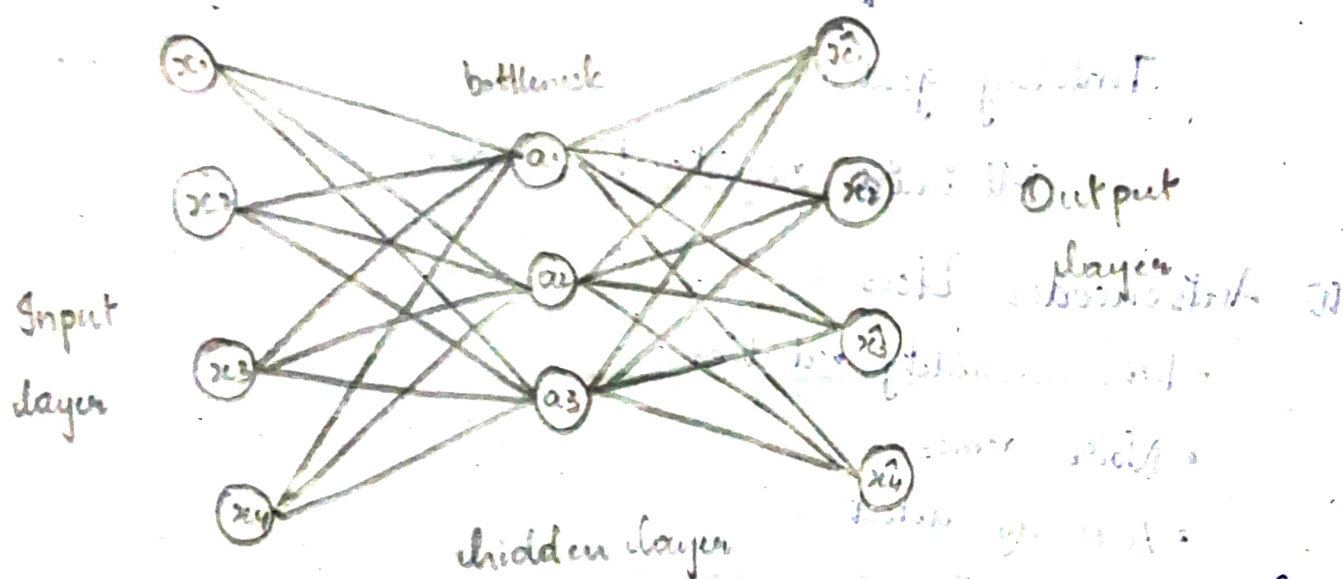
$$\tilde{h}_t = \tanh(W_h [x_t, r_t \odot h_{t-1}] + b_h)$$

Final hidden state:

$$h_t = (1 - z_t) h_{t-1} + z_t \tilde{h}_t$$

Model	Memory	Complexity	Main Point
RNN	short-term	low	Basic sequences
LSTM	long-term	High	Strong memory using 3 gates
GRU	long-term	Medium	Simpler/faster alternative to LSTM.

9. Autoencoder - A neural network trained to reconstruct its input after compressing it into a smaller internal representation.



• **Encoder**: Compresses the input into a lower-dimensional representation.

$$z = f(W_e x + b_e)$$

↓
latent representation

• **Latent Space**: A compact representation containing important information about the input.

→ original input - 100 features

↓
Encoder

↓
latent representation - 10 features

- Decoder : Reconstructs the original data from the latent representation.

$$\hat{x} = g(w_d z + b_d)$$

- Reconstruction loss :

The autoencoder compares :

original I/p vs reconstructed I/p

Common loss :

$$L = \frac{1}{n} \sum (x - \hat{x})^2$$

Training goal :

Minimize reconstruction error

10. Autoencoder Uses -

- Dimensionality reduction
- Noise removal
- Anomaly detection
- Feature learning
- Anomaly detection - Train autoencoder on normal data.

Normal I/p $\rightarrow$ low reconstruction error

Abnormal I/p $\rightarrow$ high " " "

Advantages -

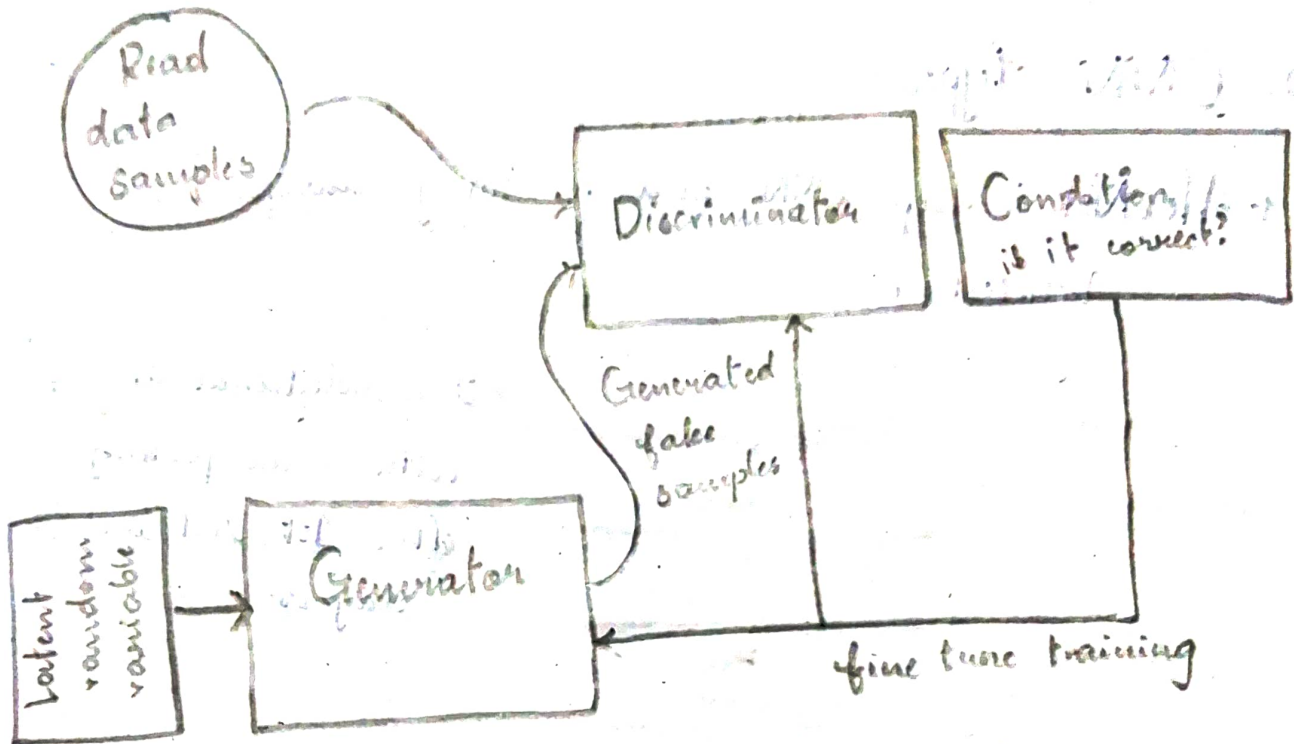
- learns useful compressed representations
- works without labels
- useful for anomaly detection

Limitations -

- can simply memorize data
- reconstruction may lose information
- latent features may be hard to interpret.

11. GAN - Generative Adversarial Network -

A generative deep-learning architecture where two networks compete - a generator creates fake data and a discriminator tries to distinguish real data from fake data.



◦ **Generator**: creates fake samples from random noise

$$x_{\text{fake}} = G(z)$$

Goal: Create data realistic enough to fool the discriminator.

◦ **Discriminator**: predicts whether a sample is real/generated.

$$\text{o/p} - D(x) \downarrow$$

represents probability of input being real

Applications -

- Image generation
- Image-to-image translation
- Super-resolution
- Synthetic data generation
- Art generation

Challenges -

- Mode collapse

Generator produces limited varieties of outputs.

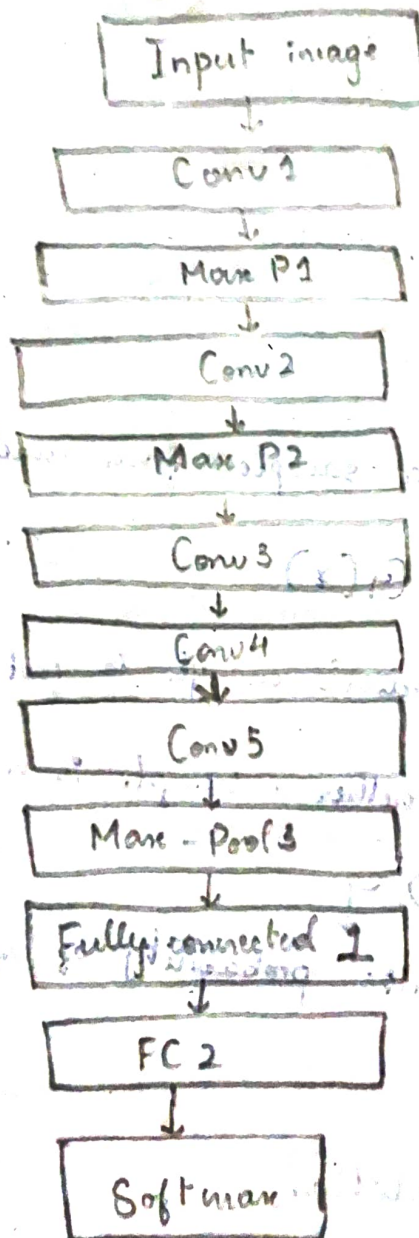
- Training instability

G and D must remain reasonably balanced.

12. CNN types -

- 1. AlexNet : deep CNN used for image classification.

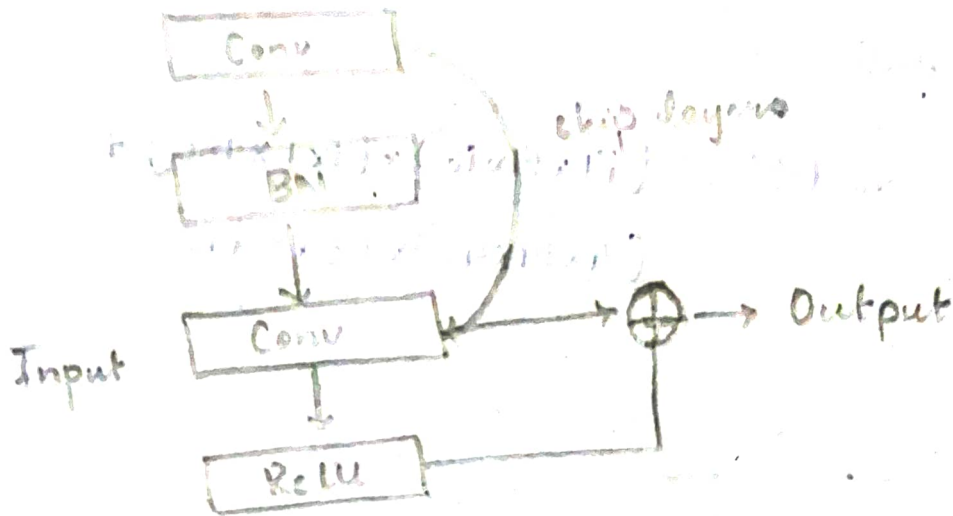
Architecture



→ 5 convolutional layers with max pooling after 1st, 2nd and 5th layers.

- Overlapping max pooling (3x3 filter, stride 2)
- 2 FC layers with dropout for regularization.
- Softmax layer for final classification output.

2. ResNet : designed to enable efficient training of very deep neural networks.
Introduces skip connections.



ResNet-34 :

- first stage - 3 residual blocks, each with 2 convolution layers of 64 filters and identity skip connections.
- 2nd stage - 4 residual blocks, each with 2 convolution layers of 128 filters and uses 1x1 projection/padding.
- 3rd stage - 6 residual blocks, each with 2 convolution layers of 256 filters.
- 4th stage - 3 residual blocks, each with 2 convolution layers of 512 filters.
- Output layer - feature maps are passed through Global Average Pooling followed by a FC layer with softmax for classification.

$$H(x) = F(x) + x \rightarrow \text{input to block}$$

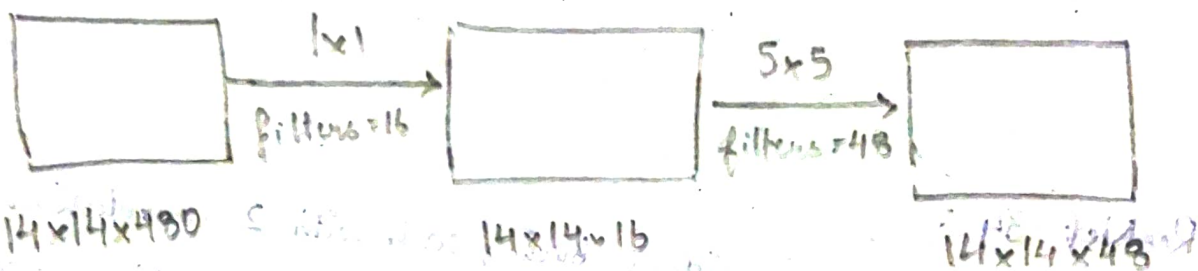
$\downarrow$ desired mapping $\downarrow$ residual function to be learned

3. GoogleNet: CNN designed for efficient image ch

→ Uses inception module to process multiple filter sizes in parallel improving feature extraction.

Key features -

- 1×1 convolution: $(14 \times 14 \times 16) \times (1 \times 1 \times 480) + (14 \times 14 \times 48) \times (5 \times 5 \times 16)$



- Global average pooling: averages each feature map into a single value.

- Inception module: multiple operations in parallel

- 1×1 convolutions

- 3×3 "

- 5×5 "

- 3×3 max pooling

- Auxiliary classifiers: reduce vanishing gradients problems.

4. VGG16: Visual Geometry group at Oxford University

